IT STUDY HUB

COMPLETE R PROGRAMMING SERIES

MODULE 7: ADVANCED DATA MANIPULATION WITH TIDYVERSE

Authors:

Mithila Rani Saha & Dithsa Dutta

www.itstudyhub.dpdns.org

WELCOME TO MODULE 7

About This Module

Welcome to Module 7 of the **IT Study Hub Complete R Programming Series**. In Module 5, you learned the basic grammar of data manipulation using `dplyr`. Now, it is time to master the advanced verbs and workflows that define professional data science.

Data in enterprise environments is incredibly complex. It resides across multiple disparate databases, comes structurally mutated, and requires heavy algorithmic aggregation before it can be visualized or modeled. This module dives deep into the Tidyverse. You will learn to perform powerful relational joins, harness the Split-Apply-Combine methodology via `group_by()` and `summarise()`, and reshape architectural dimensions using the pivoting tools of `tidyr`.

IT STUDY HUB

Learning Outcomes

By the completion of this comprehensive module, you will be able to:

- Utilize advanced selection helpers and logic within `filter()`, `select()`, and `mutate()`.
- Calculate complex, grouped aggregations using the Split-Apply-Combine framework.
- Merge fragmented datasets logically using Inner, Left, Right, and Full Mutating Joins.
- Diagnose and isolate dataset anomalies using Semi and Anti Filtering Joins.
- Apply bulk mathematical transformations safely across hundreds of columns using `across()`.
- Execute dynamic conditional logic directly within data frames using `case_when()`.
- Reshape unreadable database exports into perfectly Tidy format using `pivot_longer()` and `pivot_wider()`.

Module Coverage

This handbook thoroughly covers the following core topics:

1. Advanced select() and filter()
2. Advanced mutate() and arrange()
3. group_by() and summarise()
4. Mutating Joins (Inner, Left, Right, Full)
5. Filtering Joins (Semi, Anti)
6. Bulk transformations with across()
7. Conditional logic (case_when)
8. Tidy Data: Wide vs. Long Format
9. Reshaping: pivot_longer()
10. Reshaping: pivot_wider()
11. separate() and unite()
12. The modern pipes: %>% vs |>

IT STUDY HUB

MAIN NOTES

1. Advanced dplyr: select() and filter() Revisited

CONCEPT:
While select() isolates columns and filter() isolates rows, advanced usage involves helper functions and complex logic to manipulate massive datasets instantly.

Deep Dive Explanation:
In Module 5, you learned basic subsets. Advanced `dplyr` unlocks powerful helper functions. Inside `select()`, you can use `starts_with()`, `ends_with()`, or `contains()` to grab hundreds of columns dynamically without typing their names. Inside `filter()`, you can use `%in%` to check if a row matches a list of values, or use `is.na()` to filter out (or specifically keep) missing data.

Real-Life Example:
Selecting all columns related to Q4 revenue out of a 500-column dataset by simply typing `select(data, ends_with("_Q4"))`.

Syntax Structure:

```
select(df, starts_with("text"))
filter(df, col_name %in% c("A", "B"))
```

R Code Example

```
library(dplyr)

df <- tibble(
  emp_id = 1:4,
  dept = c("IT", "HR", "IT", "Sales"),
  rev_2023 = c(100, 0, 150, 200),
  rev_2024 = c(120, 0, 160, 210)
)

# Advanced select & filter
advanced_df <- df %>%
  select(emp_id, starts_with("rev")) %>%
  filter(dept %in% c("IT", "Sales"))

print(advanced_df)
```

Console / Visual Output

```
# A tibble: 3 × 3
  emp_id rev_2023 rev_2024

1      1      100      120
2      3      150      160
3      4      200      210
```

Line-by-Line Explanation:

The `starts_with("rev")` instantly grabbed both revenue columns without us naming them explicitly. `dept %in% c("IT", "Sales")` is a shortcut for writing `dept == "IT" | dept == "Sales"`, gracefully dropping the HR employee.

Real-World Use:

When parsing medical data, a researcher might use `select(contains("blood_pressure"))` to isolate 20 variations of a metric scattered across a massive clinical trial dataset.

Common Mistakes:

Forgetting that `select()` is strictly for Columns and `filter()` is strictly for Rows. You cannot `select(Age > 18)`.

Best Practices:

Always use `%in%` instead of chaining multiple `==` statements with `|` (OR) when checking against 3 or more categories. It is significantly more readable.

Practice Challenge:

Write a command to select all columns that end with `"_date"`.

Comparison: select vs filter

Feature	`select()`	`filter()`
Action	Picks Columns	Picks Rows
Logic Type	Names / Column Positions	Logical Evaluations (==, >)
Helpers	`starts_with()`, `contains()`	`%in%`, `is.na()`

2. Advanced dplyr: mutate() and arrange()

CONCEPT:
mutate() calculates and appends new columns, while arrange() handles complex, multi-tiered data sorting.

Deep Dive Explanation:
Advanced `mutate()` allows you to overwrite existing columns or create rolling calculations (like cumulative sums). `arrange()` can sort by multiple columns sequentially. If the first column has a tie, it looks at the second column to break the tie, and so on. You can mix ascending and descending sorts in a single line.

Real-Life Example:
Creating a new column representing Profit Margin using `mutate()`, then using `arrange()` to sort by Department alphabetically, and within each Department, sorting by the highest Profit Margin.

Syntax Structure:

```
mutate(df, new_col = A / B)
arrange(df, col1, desc(col2))
```

R Code Example

```
library(dplyr)

sales <- tibble(
  region = c("North", "North", "South", "South"),
  product = c("A", "B", "A", "B"),
  revenue = c(500, 800, 400, 900)
)

# Multi-tier arrange and mutate
processed <- sales %>%
  mutate(tax = revenue * 0.10) %>%
  arrange(region, desc(revenue))

print(processed)
```

Console / Visual Output

```
# A tibble: 4 × 4
  region product revenue   tax

1 North  B           800    80
2 North  A           500    50
3 South  B           900    90
4 South  A           400    40
```

Line-by-Line Explanation:

We created the `tax` column. Then we sorted by `region` (North comes before South alphabetically). Inside the 'North' ties, it sorted by `revenue` descending (800 before 500).

Real-World Use:

Financial reports frequently require data to be sorted by Year (Ascending), then by Quarter (Ascending), and finally by Revenue (Descending).

Common Mistakes:

Typing `arrange(desc(col1, col2))` instead of `arrange(desc(col1), desc(col2))`. The `desc()` function must wrap each column individually.

Best Practices:

When using `mutate()`, remember you can refer to a newly created column in the exact same `mutate()` call to create yet another column.

Practice Challenge:

Write code to sort a dataframe by `Age` (Ascending) and `Score` (Descending).

Comparison: mutate vs summarise

Feature	`mutate()`	`summarise()`
Row Count	Stays EXACTLY the same.	Collapses into ONE row (per group).
Purpose	Calculate metrics for each individual record.	Calculate aggregate metrics for the entire set.
Example	`Profit = Revenue - Cost`	`Total_Profit = sum(Revenue)`

3. Aggregation: summarise() and group_by()

CONCEPT:
The split-apply-combine workflow allows you to calculate summary statistics for independent groups within your data.

Deep Dive Explanation:
If `mutate()` creates a new column while keeping the same number of rows, `summarise()` destroys the original rows and collapses the dataset into a single summary row (like a total sum or average). However, `summarise()` is rarely used alone. It is usually paired with `group_by()`. `group_by()` invisible 'splits' the data into chunks based on a category. When `summarise()` is applied next, it calculates the summary metric *for each specific chunk*.

Real-Life Example:
Instead of finding the average salary of the entire company, `group_by(Department)` followed by `summarise(mean(Salary))` outputs a small table showing the specific average salary for IT, HR, and Sales separately.

Syntax Structure:

```
df %>% group_by(category) %>% summarise(metric = mean(value))
```

R Code Example

```
library(dplyr)

emp <- tibble(
  dept = c("IT", "IT", "HR", "HR"),
  salary = c(60, 80, 50, 55)
)

# Split-Apply-Combine
dept_summary <- emp %>%
  group_by(dept) %>%
  summarise(
    avg_salary = mean(salary),
    emp_count = n()
  )

print(dept_summary)
```

Console / Visual Output

```
# A tibble: 2 × 3
  dept  avg_salary emp_count

1 HR          52.5         2
2 IT          70           2
```

Line-by-Line Explanation:

`group_by(dept)` split the 4 rows into two groups (HR and IT). `summarise()` collapsed them, calculating the `mean()` salary and counting the number of rows `n()` per group.

Real-World Use:

This is the exact R equivalent of an Excel Pivot Table. It is the most common operation in daily data analysis.

Common Mistakes:

Forgetting to `ungroup()` after complex group operations. While `summarise()` drops one grouping level automatically, if you `mutate()` a grouped dataframe, the invisible groupings persist and can corrupt downstream math.

Best Practices:

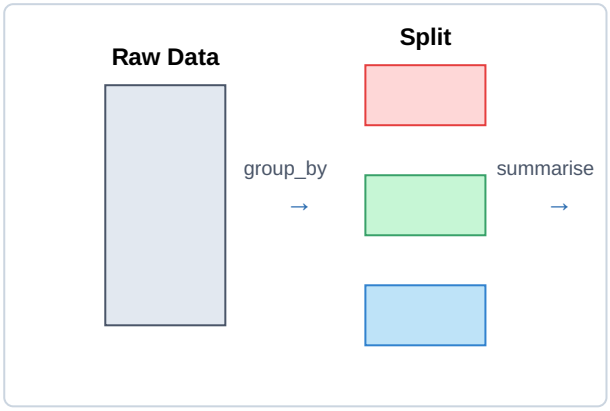
Always use `n()` alongside your summary metrics (like mean/median) so you know exactly how many observations generated that metric. An average based on 2 people is very different from an average based on 2,000.

Practice Challenge:

Write a pipeline to group by `Country` and calculate the `max(Revenue)`.

The Split-Apply-Combine Workflow

How `group_by()` perfectly routes data into `summarise()`.



4. Combining Datasets: Mutating Joins

CONCEPT:
Mutating joins merge two separate data frames together based on a common matching column (a key).

Deep Dive Explanation:
In business, data is scattered. Customer names are in Table A; their purchase histories are in Table B. To analyze them together, you must 'join' them using a matching key (like `Customer_ID`). 1. **inner_join()**: Keeps ONLY rows that exist in BOTH tables. 2. **left_join()**: Keeps ALL rows from the Left table, bringing in matching data from the Right table. (Fills with NA if no match). 3. **right_join()**: Keeps ALL rows from the Right table. 4. **full_join()**: Keeps EVERYTHING, merging matches and filling non-matches with NAs.

Real-Life Example:
You have a list of all 100 employees (Table A). You have a list of 80 employees who took a training course (Table B). A `left_join(A, B)` keeps all 100 employees, adding the training data to the 80 who took it, and putting `NA` for the 20 who didn't.

Syntax Structure:

```
left_join(x, y, by = "key_column")
```

R Code Example

```
library(dplyr)

customers <- tibble(id = c(1, 2, 3), name = c("Ali", "Bob", "Cat"))
orders <- tibble(id = c(1, 2, 4), item = c("PC", "Mouse", "Desk"))

# Left Join: Keep all customers
result_left <- left_join(customers, orders, by = "id")
print(result_left)

# Inner Join: Keep only matching pairs
result_inner <- inner_join(customers, orders, by = "id")
print(result_inner)
```

Console / Visual Output

```
# A tibble: 3 × 3
     id name  item

1     1 Ali   PC
2     2 Bob   Mouse
3     3 Cat   NA

# A tibble: 2 × 3
     id name  item

1     1 Ali   PC
2     2 Bob   Mouse
```

Line-by-Line Explanation:

In `left_join`, 'Cat' (id=3) stays, but her item is NA because she has no orders. In `inner_join`, 'Cat' is completely dropped, and the 'Desk' (id=4) from the orders table is also dropped because neither had a match in the opposite table.

Real-World Use:

`left_join()` is the most common join in data science. It acts exactly like a VLOOKUP in Excel, pulling supplementary data into your primary master table.

Common Mistakes:

Joining on columns with different data types (e.g., `id` is Numeric in Table A but Character in Table B). R will throw an error. Coerce them to match first using `as.character()` or `as.numeric()`.

Best Practices:

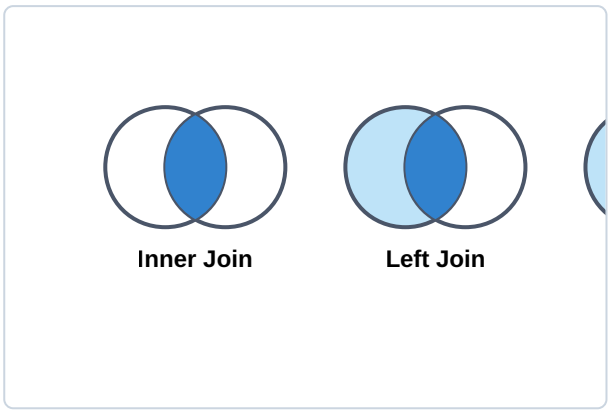
Always explicitly define the `by = "column_name"` argument to ensure R doesn't accidentally join on a completely unrelated column that happens to share the same name (like `Date`).

Practice Challenge:

Write the syntax to join `df1` and `df2` keeping only rows present in BOTH dataframes.

IT STUDY

Mutating Join Logic



Comparison: Inner vs Left vs Right vs Full Join

Join Type	Left Table (x)	Right Table (y)	NAs Generated?
Inner	Keeps ONLY matches	Keeps ONLY matches	No
Left	Keeps ALL rows	Keeps ONLY matches	Yes (If missing in Y)
Right	Keeps ONLY matches	Keeps ALL rows	Yes (If missing in X)
Full	Keeps ALL rows	Keeps ALL rows	Yes (For all non-matches)

5. Filtering Joins: Semi and Anti Joins

CONCEPT:
Filtering joins do NOT add new columns. They simply filter the rows of the first table based on the existence of matches in the second table.

Deep Dive Explanation:
Sometimes you don't want to bring in new data; you just want to filter your list. - **semi_join()**: Keeps rows in Table A that HAVE a match in Table B. - **anti_join()**: Keeps rows in Table A that DO NOT HAVE a match in Table B.

Real-Life Example:
You have an email list of 10,000 subscribers (Table A). You have a list of 500 people who 'Unsubscribed' (Table B). An `anti_join(A, B)` instantly returns the 9,500 active subscribers who are NOT on the unsubscribe list.

Syntax Structure:

```
anti_join(x, y, by = "key_column")
```

R Code Example

```
library(dplyr)

active_users <- tibble(user_id = 1:4, user = c("A", "B", "C", "D"))
banned_users <- tibble(user_id = c(2, 4))

# Keep users who are NOT banned
safe_users <- anti_join(active_users, banned_users, by = "user_id")
print(safe_users)

# Keep users who ARE banned (but don't add columns from the banned
table)
bad_users <- semi_join(active_users, banned_users, by = "user_id")
print(bad_users)
```

Console / Visual Output

```
# A tibble: 2 × 2
  user_id user

1       1 A
2       3 C

# A tibble: 2 × 2
  user_id user

1       2 B
2       4 D
```

Line-by-Line Explanation:

The `anti_join` removed users 2 and 4. The `semi_join` kept ONLY users 2 and 4. Notice that no new columns were added to the outputs.

Real-World Use:

`anti_join` is the absolute fastest way to find anomalies. If you have an 'Expected Transactions' table and an 'Actual Transactions' table, an `anti_join` immediately isolates the missing transactions.

Common Mistakes:

Confusing `semi_join` with `inner_join`. `inner_join` attaches columns from the right table. `semi_join` purely filters the left table and ignores the right table's columns entirely.

Best Practices:

Use `anti_join()` as a diagnostic tool. After performing a `left_join`, run an `anti_join` to see exactly which rows failed to find a match.

Practice Challenge:

Write the command to find all records in `employees` that are missing from `payroll`.

Comparison: semi_join vs anti_join

Join Type	Action on Left Table	Adds Columns from Right?
semi_join	Keeps rows with a Match.	NO. It purely filters.
anti_join	Keeps rows WITHOUT a Match.	NO. It purely isolates anomalies.

6. Advanced Transformations: across()

CONCEPT:
The across() function allows you to apply the same transformation or summary to multiple columns simultaneously without repeating code.

Deep Dive Explanation:
If you need to round 50 numeric columns to 2 decimal places, writing 50 `mutate` statements is exhausting. In modern `dplyr`, `across()` solves this. You specify *which* columns to target (e.g., `where(is.numeric)`), and *what* function to apply to them (e.g., `round`).

Real-Life Example:
Selecting a dataframe containing 20 text columns and converting all of them to uppercase letters instantly using `mutate(across(where(is.character), toupper))`.

Syntax Structure:

```
mutate(across(columns_to_target, function_to_apply))
```

R Code Example

```
library(dplyr)

data <- tibble(
  id = 1:2,
  price1 = c(10.555, 20.123),
  price2 = c(100.987, 200.456)
)

# Round ALL numeric columns to 1 decimal place
rounded_data <- data %>%
  mutate(across(where(is.numeric), ~ round(.x, 1)))

print(rounded_data)
```

Console / Visual Output

```
# A tibble: 2 × 3
     id price1 price2

1     1   10.6   101
2     2   20.1   200.5
```

Line-by-Line Explanation:

`across()` scans the dataframe. `where(is.numeric)` identifies `id`, `price1`, and `price2`. The tilde `~` creates a temporary function where `.x` represents the column being modified, dynamically rounding every numeric column to 1 decimal place.

Real-World Use:

Normalizing data for machine learning. You can scale 1,000 numeric features from 0 to 1 simultaneously using a single `mutate(across(...))` statement.

Common Mistakes:

Forgetting the tilde `~` or `.x` when trying to pass arguments (like the number of decimal places) to the function inside `across`.

Best Practices:
You can target columns by name prefixes instead of data type: `across(starts_with("price"), mean)`.

Practice Challenge:
Write a command using `across()` to convert all columns `where(is.character)` to a factor using `as.factor`.

7. Conditional Transformations (case_when & if_else)

CONCEPT:
These functions execute complex conditional logic entirely inside data frame pipelines without breaking the vectorized flow.

Deep Dive Explanation:
We covered base R's `ifelse()` in Module 4. `dplyr` provides two upgraded versions. `if_else()` is a stricter, safer version of `ifelse()` that strictly enforces data type matching. `case_when()` is a massive upgrade that allows you to chain an unlimited number of conditions beautifully without writing messy, nested `if_else` statements.

Real-Life Example:
Categorizing test scores: >= 90 is A, >= 80 is B, >= 70 is C, and everything else is F. `case_when()` maps this logic directly into a single new column.

Syntax Structure:

```
case_when(
  condition1 ~ "Result 1",
  condition2 ~ "Result 2",
  TRUE ~ "Default Result"
)
```

R Code Example

```
library(dplyr)

scores_df <- tibble(name = c("Ali", "Bob", "Cat"), score = c(95, 75,
40))

graded_df <- scores_df %>%
  mutate(grade = case_when(
    score >= 90 ~ "A",
    score >= 80 ~ "B",
    score >= 70 ~ "C",
    TRUE        ~ "F"  # The 'TRUE' acts as the final 'else'
  ))

print(graded_df)
```

Console / Visual Output

```
# A tibble: 3 × 3
  name  score grade

1 Ali      95 A
2 Bob      75 C
3 Cat      40 F
```

Line-by-Line Explanation:

Inside `mutate()`, `case_when()` evaluated each row top-to-bottom. Ali hit the first condition (`>=90`). Bob failed the first two but hit the third (`>=70`). Cat failed all three, falling into the universal `TRUE` catch-all.

Real-World Use:

Risk assessment modeling. Assigning customers a 'High', 'Medium', or 'Low' risk tier based on multiple dynamic combinations of age, credit score, and income.

Common Mistakes:

Forgetting the `TRUE ~ "Default"` line at the very end. If a row fails all your specific conditions and there is no default fallback, `case_when()` will silently output `NA` for that row.

Best Practices:

Ensure the results on the right side of the tilde `~` are exactly the same data type. You cannot return the number `1` for the first condition and the string `"Fail"` for the second.

Practice Challenge:

Use `case_when` to create a `Size` column: if `x < 10` ~ "Small", `x < 20` ~ "Med", else ~ "Large".

8. The Concept of Tidy Data: Wide vs. Long Format

CONCEPT:

Data exists in two fundamental shapes: Wide (matrix-like, easy for humans to read) and Long (relational, required by computers).

Deep Dive Explanation:

Humans prefer **Wide Data**. In a wide format, subjects are rows, and each time period (e.g., 2021, 2022, 2023) gets its own column. It looks like a classic spreadsheet. Computers and plotting software like `ggplot2` require **Long Data**. In Long data, 'Year' becomes a single column, and the values get stacked vertically. This fulfills the Tidy Data rule: 'Every variable (Year, Value) must be its own column.'

Real-Life Example:
Wide: A hospital spreadsheet with columns for 'Jan Weight', 'Feb Weight', 'Mar Weight'. Long: The same data reshaped into just three columns: 'Patient ID', 'Month', and 'Weight'.

Syntax Structure:

```
No specific syntax; it is an architectural philosophy.
```

R Code Example

```
# WIDE DATA (Untidy for plotting)
wide_df <- tibble(ID = 1, `2023` = 50, `2024` = 60)

# LONG DATA (Tidy for plotting)
long_df <- tibble(ID = c(1, 1), Year = c(2023, 2024), Value = c(50, 60))

print(wide_df)
print(long_df)
```

Console / Visual Output

```
# WIDE (2023 & 2024 are Columns)
# A tibble: 1 × 3
     ID `2023` `2024`

1     1     50     60

# LONG (Year & Value are Columns)
# A tibble: 2 × 3
     ID  Year Value

1     1  2023    50
2     1  2024    60
```

Line-by-Line Explanation:

Notice how the Long version has more rows but fewer columns. The column headers '2023' and '2024' from the wide format were melted down into actual data values under the new 'Year' column.

Real-World Use:

To plot a multi-colored line graph tracking revenue over time for 5 different products in `ggplot2`, the data MUST be in long format so 'Product' can be mapped to `color` and 'Year' mapped to the `x` axis.

Common Mistakes:

Trying to write a massive `ggplot` or complex `group_by` function using Wide Data. You will end up writing dozens of lines of repetitive code.

Best Practices:

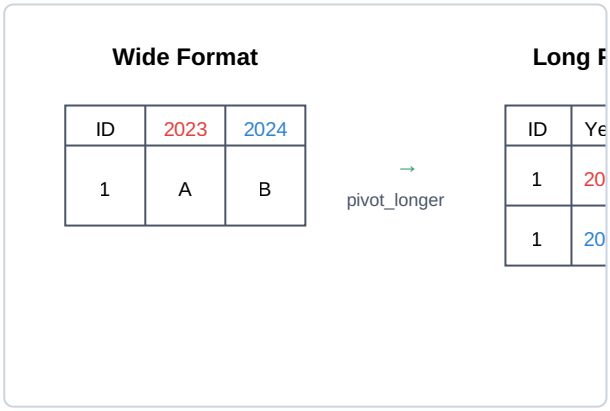
Store and share data with humans in Wide format. The very first step of your R analysis script should be converting it to Long format.

Practice Challenge:

Identify whether a spreadsheet of weather data with columns `City`, `Monday_Temp`, `Tuesday_Temp`, `Wednesday_Temp` is Wide or Long.

Wide Data vs Long Data (Tidy)

Machine learning and visualization require Long format.



9. Reshaping Data: pivot_longer()

CONCEPT:
The pivot_longer() function from the tidyr package automatically converts Wide data into computationally powerful Long data.

Deep Dive Explanation:
To fix untidy Wide data, you use `pivot_longer()`. You tell the function exactly which columns are violating the rules (e.g., the columns representing years). R then takes those column headers, melts them down into a new 'names' column (e.g., 'Year'), and stacks all their intersecting numbers into a new 'values' column (e.g., 'Revenue').

Real-Life Example:
Melting columns 'Q1', 'Q2', 'Q3', 'Q4' into a single column called 'Quarter', and moving the dollar amounts into a single column called 'Sales'.

Syntax Structure:

```
pivot_longer(cols = columns_to_melt, names_to = "new_name_col",
values_to = "new_val_col")
```

R Code Example

```
library(tidyr)
library(dplyr)

wide_sales <- tibble(
  store = c("NY", "LA"),
  Q1 = c(100, 200),
  Q2 = c(110, 210)
)

# Melting Q1 and Q2 columns into Long format
long_sales <- wide_sales %>%
  pivot_longer(cols = c(Q1, Q2),
               names_to = "Quarter",
               values_to = "Revenue")

print(long_sales)
```

Console / Visual Output

```
# A tibble: 4 × 3
  store Quarter Revenue

1 NY    Q1          100
2 NY    Q2          110
3 LA    Q1          200
4 LA    Q2          210
```

Line-by-Line Explanation:

We kept the `store` column fixed. We told `pivot_longer` to grab columns Q1 and Q2. It took the headers (Q1, Q2) and stuffed them into the new `Quarter` column. It took the numbers (100, 110, etc.) and stacked them into the `Revenue` column.

Real-World Use:

You receive a government demographics spreadsheet where every year from 1950 to 2020 has its own column. You use `pivot_longer(cols = '1950':'2020', names_to = 'Year', values_to = 'Population')`.

Common Mistakes:

Forgetting to wrap numeric column headers in backticks (e.g., `` `2020` ``) when identifying which columns to melt.

Best Practices:

Instead of typing out all 50 columns you want to melt, use negative selection: `cols = -store`. This tells R to melt everything EXCEPT the store column.

Practice Challenge:

Write the command to pivot columns `Jan`, `Feb`, and `Mar` into `names_to = "Month"` and `values_to = "Sales"`.

10. Reshaping Data: pivot_wider()

CONCEPT:

The pivot_wider() function converts Long data back into human-readable Wide data by spreading rows across multiple columns.

Deep Dive Explanation:

This is the exact inverse of `pivot_longer`. Once your statistical analysis or `dplyr` pipeline is finished, your boss might ask to see the results in a classic summary table. `pivot_wider()` takes a column containing category names (e.g., 'Quarter') and spreads them horizontally to become new column headers. It then takes a values column (e.g., 'Revenue') and fills the grid.

Real-Life Example:
Taking your final Tidy dataset of 'Store', 'Quarter', and 'Revenue', and spreading it out so Q1, Q2, Q3, and Q4 each have their own column for a final presentation slide.

Syntax Structure:

```
pivot_wider(names_from = name_column, values_from = value_column)
```

R Code Example

```
library(tidyr)

# Starting with Long data
long_data <- tibble(
  student = c("Ali", "Ali", "Bob", "Bob"),
  subject = c("Math", "Sci", "Math", "Sci"),
  score = c(95, 90, 85, 80)
)

# Spreading into Wide format
wide_data <- long_data %>%
  pivot_wider(names_from = subject,
              values_from = score)

print(wide_data)
```

Console / Visual Output

```
# A tibble: 2 × 3
  student  Math   Sci

1 Ali        95    90
2 Bob        85    80
```

Line-by-Line Explanation:
The unique values in the `subject` column ("Math", "Sci") were ripped out and turned into brand new column headers. The numbers from the `score` column were populated perfectly under their respective new headers for each student.

Real-World Use:
Generating a cross-tabulation (Pivot Table) of Total Sales categorized by Region (Rows) and Product Category (Columns) to paste into an executive PowerPoint.

Common Mistakes:
Trying to use `pivot_wider` when you have duplicate identifiers. If Bob took Math twice and got two different scores, `pivot_wider` won't know which score to put in the 'Bob x Math' cell and will generate an unreadable nested list warning.

Best Practices:
Only use `pivot_wider()` at the absolute end of your data pipeline, purely for presentation formatting. Do not perform math or visualization on wide data.

Practice Challenge:
Write the command to take a `Metric` column and spread it across the headers, filling the cells with data from the `Value` column.

11. Splitting and Combining Columns (separate & unite)

CONCEPT:
The separate() and unite() functions handle dirty text fields by slicing one column into multiple, or pasting multiple columns into one.

Deep Dive Explanation:

Often, two distinct pieces of information are trapped in a single column, violating Tidy data rules (e.g., a column named `DateTime` containing '2024-01-15 14:30:00'). The `separate()` function slices the column based on a specific delimiter (like a space or hyphen). Conversely, `unite()` acts like `paste()`, fusing multiple columns together into a single unified column.

> **Real-Life Example:**
> Splitting a `Full_Name` column into `First_Name` and `Last_Name` columns based on the space between them.

Syntax Structure:

```
separate(col, into = c("new_col1", "new_col2"), sep = "_")
unite(col_name, col1, col2, sep = "_")
```

R Code Example

```
library(tidyr)
library(dplyr)

raw <- tibble(id = 1:2, geo = c("NY_USA", "London_UK"))

# Splitting 'geo' into City and Country based on the underscore
split_df <- raw %>%
  separate(geo, into = c("City", "Country"), sep = "_")
print(split_df)

# Re-uniting them with a comma
united_df <- split_df %>%
  unite("Location", City, Country, sep = ", ")
print(united_df)
```

Console / Visual Output

```
# A tibble: 2 × 3
     id City   Country

1     1 NY     USA
2     2 London UK

# A tibble: 2 × 2
     id Location

1     1 NY, USA
2     2 London, UK
```

Line-by-Line Explanation:

`separate()` targeted `geo`, looked for `_`, and created two new columns. `unite()` targeted `City` and `Country`, destroyed them, and replaced them with a single `Location` column joined by `, `.

Real-World Use:

Extracting the Area Code from a massive database of Phone Numbers: `separate(phone, into = c("AreaCode", "Number"), sep = "-")`.

Common Mistakes:

Forgetting that `separate()` deletes the original column by default. Add `remove = FALSE` if you want to keep the original raw text column as a backup.

Best Practices:

When separating dates like `2024-05-12`, use `sep = "-"`, and ensure you assign three names to the `into` argument: `c("Year", "Month", "Day")`.

Practice Challenge:

Write the command to separate a column `IP_Address` based on the period `.` delimiter.

12. The Pipes: %>% vs |> and Data Pipelines

CONCEPT:
Data pipelines string massive sequences of actions together. R now supports two types of pipes: the classic %>% and the modern native |>.

Deep Dive Explanation:
You learned the `magrittr` pipe `%>%` in Module 5. Because pipelines became so universally loved by the R community, the creators of R built a 'native' pipe directly into the core language starting in R 4.1.0: `|>`. They both do the exact same thing (taking the object on the left and passing it to the right). The native pipe `|>` is slightly faster because it doesn't require loading the `dplyr` package, though `%>%` has some advanced specific features.

Real-Life Example:
Using `|>` to import data, filter it, mutate it, and plot it in one continuous, readable flow of logic.

Syntax Structure:

```
data |> function1() |> function2()
```

R Code Example

```
library(dplyr)

# Traditional nested code (Hard to read)
# round(mean(mtcars$mpg), 1)

# Using the classic magrittr pipe
result1 <- mtcars$mpg %>%
  mean() %>%
  round(1)

# Using the modern Native R pipe (R 4.1+)
result2 <- mtcars$mpg |>
  mean() |>
  round(1)

print(paste(result1, result2))
```

Console / Visual Output

```
[1] "20.1 20.1"
```

Line-by-Line Explanation:

Instead of `round(mean(x), 1)`, the data `mpg` flows into `mean()`, and the result of that flows into `round()`. The syntax reads exactly like English: 'Take the data, THEN find the mean, THEN round it.'

Real-World Use:

Every single modern data analytics script uses pipes. It converts inside-out algebraic code into a logical, top-to-bottom recipe format.

Common Mistakes:

Trying to use the native pipe `|>` on an ancient version of R. It was introduced recently. If you work on legacy corporate servers, you may be forced to stick with `%>%`.

Best Practices:
Start migrating to the native `|>` pipe for new projects. It requires zero package dependencies and parses natively in C, making it universally reliable across all R environments.

Practice Challenge:
Convert `sqrt(sum(c(9, 16)))` into a pipeline format using `|>`.

Understanding The Pipe

```
x %>% f(y)
```

is mathematically identical to

```
f(x, y)
```

Comparison: %>% vs |>

Feature	Magrittr Pipe %>%	Native Pipe |>
Origin	`dplyr` / `magrittr` package	Base R Core Language (v4.1+)
Dependency	Requires `library(dplyr)`	Requires ZERO packages.
Speed	Fast	Slightly Faster (C-level execution)

HANDS-ON PROJECT

IT STUDY HUB

BUSINESS & SALES DATA ANALYSIS

Problem Statement:
You are the lead Data Analyst for a retail company. You have been given two messy datasets: a master 'Customers' table, and an untidy 'Sales' table spanning two years. Your objective is to join the tables securely, clean the data using conditional transformations, reshape it into a perfectly Tidy format, and aggregate the data to find the total revenue generated by high-tier customers per year. You must execute this entirely inside a seamless `dplyr` pipeline.

1. The Raw Data (Setup)

```
library(dplyr)
library(tidyr)

# Dataset 1: Customer Metadata
customers <- tibble(
  cust_id = c(101, 102, 103, 104),
  name = c("Alice", "Bob", "Charlie", "Diana"),
  status = c("Active", "Inactive", "Active", "Active")
)

# Dataset 2: Untidy Sales Data (Years are columns!)
sales <- tibble(
  customer_id = c(101, 102, 103, 105), # Notice 105 is an
anomaly!
  `2023_Revenue` = c(500, 0, 1200, 300),
  `2024_Revenue` = c(600, 0, 1500, 350)
)
```

2. Diagnostic Checks (Anti-Join)

```
# Who is in the sales table but NOT in our customer database?
anomalies <- anti_join(sales, customers, by = c("customer_id" =
"cust_id"))
print(anomalies)
# Output shows customer 105. We will drop them using a
left_join.
```

3. The Master Tidyverse Pipeline

```
# Process, Join, Pivot, and Aggregate
final_report <- customers |>

  # a. Filter out Inactive customers immediately
  filter(status == "Active") |>

  # b. Left Join the sales data to our verified active
customers
  left_join(sales, by = c("cust_id" = "customer_id")) |>

  # c. Reshape the untidy Wide data into Tidy Long data
  pivot_longer(
    cols = ends_with("Revenue"),
    names_to = "Year",
    values_to = "Revenue"
  ) |>

  # d. Clean the new 'Year' column (remove the "_Revenue" text)
  separate(Year, into = c("Year", "Metric"), sep = "_") |>
  select(-Metric) |>  # Drop the useless 'Metric' column

  # e. Conditional Transformation: Assign Customer Tiers based
on Revenue
  mutate(
    Tier = case_when(
      Revenue >= 1000 ~ "Gold",
      Revenue >= 500  ~ "Silver",
      TRUE            ~ "Bronze"
    )
  ) |>

  # f. Group By & Summarise: Total revenue per Year for Gold/
Silver
  filter(Tier %in% c("Gold", "Silver")) |>
  group_by(Year, Tier) |>
  summarise(
    Total_Revenue = sum(Revenue),
    Total_Customers = n(),
```

```
    .groups = "drop" # Drops grouping structure to prevent
downstream bugs
  ) |>

  # g. Final Sorting
  arrange(Year, desc(Total_Revenue))

print(final_report)
```

Expected Console Output

```
# A tibble: 4 × 4
  Year  Tier   Total_Revenue Total_Customers

1 2023  Gold            1200               1
2 2023  Silver           500               1
3 2024  Gold            1500               1
4 2024  Silver           600               1
```

Step-by-Step Analysis & Business Insights:

The pipeline is a masterclass in relational data wrangling. We started by diagnosing an anomaly (Customer 105) using `anti_join`. Inside the pipe, we securely merged the data using `left_join`, guaranteeing rogue sales were dropped. The crucial step was `pivot_longer`, which melted the clumsy `2023` and `2024` columns into a proper mathematical format. We sliced the text strings securely with `separate()`. Finally, we dynamically tiered the customers using `case_when()`, and executed a Split-Apply-Combine aggregation to generate the final business report.

Extension Challenge:

1. Replace the `left_join` with an `inner_join`. Does the final output change? Why or why not?
2. Use `pivot_wider()` on the `final_report` to spread the `Tier` column horizontally, creating a "Gold" column and a "Silver" column.

ASSESSMENTS & INTERVIEW PREP

15+ Practice Questions

Test your fundamental knowledge of advanced data manipulation.

1. What helper function inside `select()` grabs all columns starting with 'Q'?
2. If you want to filter rows where the `status` is exactly 'A', 'B', or 'C', what is the cleanest operator to use?
3. What function creates a new column while preserving the exact row count of the dataframe?
4. What function collapses a dataframe into a single row of summary statistics?
5. Which join keeps ALL rows from the left table and matches data from the right table?
6. Which join keeps ONLY rows that exist perfectly in both tables?
7. Which function purely filters the left table to keep rows that DO NOT have a match in the right table?
8. What `dplyr` function dynamically transforms 50 numeric columns simultaneously without writing 50 lines of code?
9. What is the ultimate fallback condition you must write at the end of a `case_when()` block?
10. Does `ggplot2` require Wide data or Long data?
11. Which `tidyr` function melts multiple columns down into two columns (names and values)?
12. Which `tidyr` function slices a single text column into multiple columns based on a delimiter?
13. What is the keyboard symbol for the Base R Native pipe?
14. Why must you type `.groups = 'drop'` at the end of a `summarise()` call?

IT STUDY HUB

15. If table A has 10 rows and table B has 10 rows, how many rows could an `inner_join` output if there are no matches?

20+ Viva Questions (With Answers)

These questions are designed for rapid-fire university viva or oral examinations.

Q1: What is the Tidyverse?

Answer: A collection of R packages designed for data science that share an underlying design philosophy and grammar (e.g., dplyr, tidyr, ggplot2).

Q2: How does `group_by()` affect a dataset?

Answer: It doesn't change how the data looks; it assigns invisible group labels so subsequent math functions operate on the chunks independently.

Q3: What is the Split-Apply-Combine methodology?

Answer: Splitting data into groups, applying a summary function to each group, and combining the results into a new table.

Q4: What is a Mutating Join?

Answer: A join that physically adds columns from one table to another (Inner, Left, Right, Full).

Q5: What is a Filtering Join?

Answer: A join that filters the rows of the left table based on the right table, without adding any new columns (Semi, Anti).

Q6: If `left_join()` cannot find a match in the right table, what does it do?

Answer: It fills the newly created columns with `NA` for that specific row.

Q7: What does `anti_join()` return?

Answer: Rows in the left table that DO NOT have a corresponding match key in the right table.

Q8: Why is `case_when()` superior to `ifelse()`?

Answer: It allows you to cleanly chain an unlimited number of conditions without creating unreadable, nested if-else blocks.

IT STUDY HUB

Q9: What happens if a row fails all conditions in a `case_when()` block?

Answer: It returns `NA` unless a universal `TRUE ~ "Default"` fallback is specified.

Q10: What is Wide Data?

Answer: Data formatted for human readability where observation attributes (like Time) are spread across multiple columns.

Q11: What is Long Data?

Answer: Data formatted for computer processing where every variable is strictly its own single column.

Q12: Which function turns Wide Data into Long Data?

Answer: `pivot_longer()`.

Q13: Which function turns Long Data into Wide Data?

Answer: `pivot_wider()`.

Q14: What arguments are mandatory for `pivot_longer()`?

Answer: `cols` (which columns to melt), `names_to` (new header column name), and `values_to` (new values column name).

Q15: What does the `separate()` function do?

Answer: It splits a single character column into multiple columns based on a specific delimiter symbol (like a space or hyphen).

Q16: What does the `unite()` function do?

Answer: It pastes multiple columns together into a single character column.

Q17: What does `across()` allow you to do?

Answer: Apply a specific function (like `mean` or `as.numeric`) to multiple columns simultaneously based on their data type or name prefix.

Q18: What is the difference between `%>%` and `|>`?

Answer: `%>%` is the older magrittr pipe requiring `dplyr`. `|>` is the native R pipe built into the core language.

Q19: What is the exact syntax to join on columns with different names?

Answer: `by = c("Left_Col" = "Right_Col")`.

IT STUDY HUB

Q20: Why should you always `ungroup()` or use `.groups = 'drop'`?

Answer: Because invisible groupings persist after summarize/mutate and can silently corrupt any future mathematical operations on the dataset.

15+ Corporate Interview Questions

Prepare for advanced data engineer/analyst technical rounds with these architecture questions.

1. Explain the Split-Apply-Combine methodology. Give an exact business scenario where `group_by()` followed by `summarise()` solves a problem.

2. You execute a `left_join` between a Customer table of 100 rows and an Orders table. The resulting table has 150 rows. Why did the row count increase?

3. Walk me through how you would use an `anti_join` as an auditing tool before officially merging two massive databases.

4. I have a dataset with 50 numeric columns and 5 character columns. I need all the numeric columns rounded to 2 decimal places. How do you do this cleanly in modern `dplyr`?

5. Explain the mathematical difference between a `left_join` and a `semi_join`.

6. When writing a `case_when()` block, why is the ordering of the conditions mathematically critical?

7. Describe a business scenario where you would intentionally choose to use `pivot_wider()` instead of `pivot_longer()`.

8. A column contains the string 'NY-Active-2024'. What `tidyr` function would you use to normalize this into Tidy format, and what parameters would you set?

9. What are the specific visual and structural differences between Wide data and Long data? Why do plotting libraries like `ggplot2` strictly require Long data?

10. Why did the R core team introduce the native `|>` pipe when the `%>%` pipe already existed?

IT STUDY HUB

11. If `summarise()` reduces a dataframe to a single row, how do you perform an aggregate calculation (like group average) but append it to the original dataframe without losing rows?

12. What is the difference between `filter(col == "A" | col == "B")` and `filter(col %in% c("A", "B"))`?

13. You use `group_by()` and `mutate()` to create a column. On the next line, you `filter()` the data, but the results are completely wrong. What invisible structural element is causing the bug?

14. When using `pivot_longer()`, how do you efficiently specify to pivot 'all columns EXCEPT the ID column'?

15. Explain how you would perform a full Outer Join (`full_join`) in R and explain how it treats non-matching records from both sides.

IT STUDY HUB

MODULE WRAP-UP

Module Summary

In Module 7, you elevated your data wrangling skills to an advanced architectural level. You mastered the intricacies of relational databases, learning to securely stitch fragmented data together using Mutating Joins (`inner_join`, `left_join`, `full_join`), and aggressively audit anomalies using Filtering Joins (`anti_join`). You unlocked the true power of `dplyr` via the Split-Apply-Combine methodology, generating complex aggregate reports with `group_by()` and `summarise()`. Furthermore, you learned to write highly efficient, bulk transformations using `across()` and dynamic conditional statements with `case_when()`. Finally, you mastered the structural reshaping of data, utilizing `tidyr` to bend unreadable Wide data into computationally flawless Long data via `pivot_longer()`. Your pipelines are now enterprise-grade.

IT STUDY HUB

Key Terms Dictionary

Term	Definition
Aggregation	Collapsing detailed data into high-level summary metrics (sums, averages).
Key	The specific column(s) shared between two tables used to join them together.
Mutating Join	A join that merges columns from a right table into a left table.
Filtering Join	A join that merely filters the left table based on the right table's existence.
across()	dplyr function used to apply the same math across multiple selected columns.
case_when()	dplyr function used to execute sequential, multi-condition logic (if/else chains).
pivot_longer()	tidyr function that melts Wide data into Tidy, Long data.
pivot_wider()	tidyr function that spreads Long data horizontally into human-readable Wide data.

Quick Revision Checklist

- ✅ Do you know the difference between an Inner Join and a Left Join?
- ✅ Can you identify when to use `semi_join` vs `anti_join` for auditing?
- ✅ Do you understand why `summarise()` must follow `group_by()`?
- ✅ Do you know how to use `%in%` inside a `filter()` command?

- ✅ Can you explain why `ggplot2` strictly prefers `pivot_longer()` data?

Advanced Coding Challenges

Challenge 1: The Auditor
You have a `transactions` table and a `flagged_accounts` table. Write a single pipeline using a filtering join to isolate all transactions that belong to flagged accounts.

Challenge 2: The Multi-Tasker
Use `mutate()` and `across()` to convert all columns ending with `"_revenue"` into numeric data types simultaneously.

Challenge 3: The Reshaper
A table has columns `Student`, `Math_Score`, `Sci_Score`. Use `pivot_longer()` to turn it into a Tidy table with columns `Student`, `Subject`, and `Score`.

WHAT COMES NEXT:

MODULE 8 — ADVANCED R PROGRAMMING

You have mastered data manipulation. In the next module, we transition into pure computer science: mastering functional programming with `purrr`, writing robust error handling (TryCatch), and diving deep into regular expressions (Regex) for text mining.